ASSIGNMENT- 06

Bachelor of Computer Applications (Hons. AI & ML)
Semester- III

Subject: Database Management System

Topic: Implementation of Data Integrity Constraints

By
Name: Mohammed Sinan nm
Reg. No.: 2511021240024

Faculty Name: RENITA JOY
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
August 2026

IMPLEMENTATION OF DATA INTEGRITY CONSTRAINTS

1. Introduction

Data integrity is an important concept in Database Management Systems (DBMS). It ensures that the data stored in a database remains accurate, consistent, valid, and reliable.

In this assignment, data integrity constraints are implemented for the Online Food Delivery System developed in the previous assignments. The database contains ten tables: Customers, Addresses, Restaurants, MenuItems, Orders, OrderItems, DeliveryAgents, Deliveries, Payments, and Reviews.

Various SQL constraints, including PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL, and DEFAULT, are used to maintain the correctness of the data and the relationships between tables.

SQL statements are also executed to verify that the constraints work correctly and prevent invalid data from entering the database.

2. Objectives

The main objectives of this assignment are:

1. To implement primary keys and foreign keys in all relevant tables.
2. To apply UNIQUE constraints to prevent duplicate values.
3. To implement CHECK constraints to validate data.
4. To use DEFAULT constraints to assign automatic values.
5. To understand the purpose of different data integrity constraints.
6. To execute SQL statements to verify data integrity.
7. To maintain consistency and reliability throughout the database.

3. Types of Data Integrity Constraints

3.1 Primary Key Constraint

A PRIMARY KEY uniquely identifies each record in a table. It does not allow duplicate or NULL values.

Example:

customer_id INT PRIMARY KEY

Purpose: Ensures that every customer has a unique identifier.

3.2 Foreign Key Constraint

A FOREIGN KEY establishes a relationship between two tables. It ensures that a referenced record exists in the parent table.

Example:

FOREIGN KEY (customer_id)

REFERENCES Customers(customer_id)

Purpose: Prevents an address from referencing a customer who does not exist.

3.3 UNIQUE Constraint

A UNIQUE constraint prevents duplicate values in a column or combination of columns.

Example:

email VARCHAR(100) UNIQUE

Purpose: Prevents two customers from registering with the same email address.

3.4 CHECK Constraint

A CHECK constraint ensures that values satisfy a specified condition.

Example:

CHECK (price >= 0)

Purpose: Prevents negative menu item prices.

3.5 NOT NULL Constraint

A NOT NULL constraint ensures that a column cannot contain a NULL value.

Example:

full_name VARCHAR(100) NOT NULL

Purpose: Ensures that every customer record contains a name.

3.6 DEFAULT Constraint

A DEFAULT constraint automatically assigns a predefined value when no value is supplied for a column.

Example:

order_status VARCHAR(20) DEFAULT 'Placed'

Purpose: Automatically assigns the status `Placed` to a new order when no status is specified.

---

4. Implementation of Data Integrity Constraints

The following SQL statements use the existing `FoodDeliveryDB` database created in Assignment 5.

Note: The commands below assume that the ten tables have already been created as described in Assignment 5. Some constraints are already included in that design; the following sections demonstrate how to inspect, verify, and add missing constraints where necessary.

4.1 Selecting the Database

USE FoodDeliveryDB;

Explanation: Selects the FoodDeliveryDB database for subsequent SQL operations.

4.2 Primary Key Constraints

Each of the ten tables has a primary key.

Table	Primary Key
Customers	customer_id

Addresses	address_id
Restaurants	restaurant_id
MenuItems	item_id
Orders	order_id
OrderItems	order_item_id
DeliveryAgents	agent_id
Deliveries	delivery_id
Payments	payment_id
Reviews	review_id

These keys uniquely identify records in their respective tables.

SQL verification:

```
SHOW KEYS FROM Customers
WHERE Key_name = 'PRIMARY';

SHOW KEYS FROM Addresses
WHERE Key_name = 'PRIMARY';

SHOW KEYS FROM Restaurants
```

```
WHERE Key_name = 'PRIMARY';

SHOW KEYS FROM MenuItems
WHERE Key_name = 'PRIMARY';

SHOW KEYS FROM Orders
WHERE Key_name = 'PRIMARY';

SHOW KEYS FROM OrderItems
WHERE Key_name = 'PRIMARY';

SHOW KEYS FROM DeliveryAgents
WHERE Key_name = 'PRIMARY';

SHOW KEYS FROM Deliveries
WHERE Key_name = 'PRIMARY';

SHOW KEYS FROM Payments
WHERE Key_name = 'PRIMARY';

SHOW KEYS FROM Reviews
WHERE Key_name = 'PRIMARY';
```

Expected Result: Each command displays the primary key defined for the specified table.

4.3 Foreign Key Constraints

Foreign keys connect related tables and maintain referential integrity.

The following relationships are implemented in the database:

Child Table	Foreign Key	Referenced Table
Addresses	customer_id	Customers
MenuItems	restaurant_id	Restaurants
Orders	customer_id	Customers
Orders	restaurant_id	Restaurants
Orders	address_id	Addresses
OrderItems	order_id	Orders
OrderItems	item_id	MenuItems
Deliveries	order_id	Orders
Deliveries	agent_id	DeliveryAgents
Payments	order_id	Orders
Reviews	order_id	Orders
Reviews	customer_id	Customers

The following query displays the foreign key constraints defined in the database.

```
SELECT
   TABLE_NAME,
   COLUMN_NAME,
   CONSTRAINT_NAME,
   REFERENCED_TABLE_NAME,
   REFERENCED_COLUMN_NAME
FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = DATABASE()
  AND REFERENCED_TABLE_NAME IS NOT NULL;
```

Explanation: The query retrieves foreign key information, including the child table, column name, referenced table, and referenced column.

Expected Result: The query lists the foreign key relationships defined in FoodDeliveryDB.

4.4 UNIQUE Constraints

UNIQUE constraints prevent duplicate values in columns where uniqueness is required.

In the Online Food Delivery System, the following constraints are appropriate:

- Customers.email must be unique.
- Customers.phone_number must be unique.
- Deliveries.order_id must be unique if each order can have only one delivery record.
- Payments.order_id must be unique if each order can have only one payment record.
- Reviews.order_id must be unique if each order can have at most one review.

The Customers table already defines UNIQUE constraints on email and phone_number.

The following commands add the remaining constraints if they have not already been defined.

```
ALTER TABLE Deliveries
ADD CONSTRAINT uq_delivery_order
UNIQUE (order_id);
```

ALTER TABLE Payments

ADD CONSTRAINT uq_payment_order

UNIQUE (order_id);

ALTER TABLE Reviews

ADD CONSTRAINT uq_review_order

UNIQUE (order_id);

Important: Execute each `ALTER TABLE` statement only if the corresponding UNIQUE constraint does not already exist. Assignment 5 already includes these constraints, so adding them again will produce an error.

Purpose: These constraints enforce the intended one-record-per-order relationships.

Verification:

SHOW INDEX FROM Customers;

SHOW INDEX FROM Deliveries;

SHOW INDEX FROM Payments;

SHOW INDEX FROM Reviews;

Expected Result: The output displays the indexes and UNIQUE constraints defined on these tables.

4.5 CHECK Constraints

CHECK constraints prevent invalid values from being inserted into the database.

The following examples demonstrate appropriate validation rules for the Online Food Delivery System.

A. Menu Item Price

CHECK (price >= 0)

This ensures that the price of a food item is not negative.

B. Order Item Quantity

CHECK (quantity > 0)

This ensures that an order item has a positive quantity.

C. Payment Amount

CHECK (amount >= 0)

This prevents negative payment amounts.

D. Restaurant Rating

CHECK (rating BETWEEN 0 AND 5)

This ensures that the restaurant rating remains within the permitted range.

E. Customer Review Ratings

CHECK (restaurant_rating BETWEEN 1 AND 5),

CHECK (delivery_rating BETWEEN 1 AND 5)

These constraints ensure that both review ratings are between 1 and 5.

F. Order Status

```
CHECK (order_status IN (
    'Placed',
    'Preparing',
    'Out for Delivery',
    'Delivered',
    'Cancelled'
))
```

This restricts order status values to the permitted options.

These CHECK constraints are already included in the table definitions provided in Assignment 5. They should not be added again if they already exist.

4.6 DEFAULT Constraints

DEFAULT constraints assign values automatically when an INSERT statement does not provide a value for a column.

Examples from the database include:

Table	Column	Default Value
Restaurants	is_active	TRUE
MenuItems	is_available	TRUE
Orders	order_status	Placed
Orders	total_amount	0.00
Deliveries	delivery_status	Assigned
Payments	payment_status	Pending
Reviews	review_date	Current date and time

For example:

order_status VARCHAR(20)

DEFAULT 'Placed'

This assigns the status `Placed` when a new order is inserted without a specified order status.

Similarly:

payment_status VARCHAR(20)

DEFAULT 'Pending'

This assigns the initial payment status as `Pending`.

Purpose: DEFAULT constraints reduce repetitive data entry and provide consistent initial values.

---

5. Verification of Data Integrity

The following SQL statements demonstrate how the constraints prevent invalid data from being inserted.

For these tests, assume that the database contains the sample records inserted in Section 6.

5.1 Testing Primary Key Integrity

The following statement attempts to insert a customer using an existing customer ID.

```
INSERT INTO Customers (
    customer_id,
    full_name,
    email,
    phone_number,
    password_hash,
    registration_date
)
VALUES (
    1,
    'Duplicate Customer',
    'duplicate@example.com',
```

```
    '9000000001',
    'sample_hash',
    '2026-10-22'
);
```

Expected Result: If customer ID 1 already exists, MySQL rejects the insertion because the primary key must be unique.

5.2 Testing UNIQUE Constraint

The following statement attempts to register another customer using an existing email address.

```
INSERT INTO Customers (
    full_name,
    email,
    phone_number,
    password_hash,
    registration_date
)
VALUES (
    'Another Customer',
    'sinan@example.com',
    '9000000002',
    'sample_hash',
    '2026-10-22'
);
```

Expected Result: If `sinan@example.com` already exists, MySQL rejects the insertion because the email column has a UNIQUE constraint.

5.3 Testing Foreign Key Integrity

The following statement attempts to create an address for a customer who does not exist.

```
INSERT INTO Addresses (
    customer_id,
    address_line,
    city,
    state,
    pincode,
    address_type
)
VALUES (
    9999,
    'Sample Street',
    'Bengaluru',
    'Karnataka',
    '560001',
    'Home'
);
```

Expected Result: MySQL rejects the insertion if customer ID 9999 does not exist in Customers.

This demonstrates that foreign keys prevent references to nonexistent parent records.

5.4 Testing CHECK Constraint

The following statement attempts to insert a menu item with a negative price.

```
INSERT INTO MenuItems (
    restaurant_id,
    item_name,
    description,
```

```
    price,
    category,
    is_available
)
VALUES (
    1,
    'Sample Burger',
    'Test item',
    -100.00,
    'Main Course',
    TRUE
);
```

Expected Result: If the CHECK constraint is enforced, MySQL rejects the insertion because the price is negative.

5.5 Testing Quantity Validation

```
INSERT INTO OrderItems (
    order_id,
    item_id,
    quantity,
    price_at_order
)
VALUES (
    1,
    1,
    0,
    100.00
```

);

Expected Result: MySQL rejects the insertion because the quantity must be greater than zero.

This test assumes that the referenced order and menu item exist.

5.6 Testing DEFAULT Constraint

The following statement creates an order without explicitly providing its status.

```
INSERT INTO Orders (
    customer_id,
    restaurant_id,
    address_id,
    total_amount
)
VALUES (
    1,
    1,
    1,
    250.00
);
```

Expected Result: If the referenced customer, restaurant, and address exist, the order is inserted with the default status `Placed`.

The generated order ID can be checked using:

```
SELECT *
FROM Orders
ORDER BY order_id DESC
LIMIT 1;
```

The result should display the newly inserted order and its default status.

---

6. Sample Data for Testing

The following statements insert sample records for testing the relationships and constraints.

Execute them in the order shown.

6.1 Insert a Customer

```
INSERT INTO Customers (
    full_name,
    email,
    phone_number,
    password_hash,
    registration_date
)
VALUES (
    'Sinan',
    'sinan@example.com',
    '9876543210',
    'sample_hash',
    '2026-10-22'
);
```

6.2 Insert a Delivery Address

```
INSERT INTO Addresses (
    customer_id,
    address_line,
```

```
    city,
    state,
    pincode,
    address_type
)
VALUES (
    1,
    'Main Road',
    'Bengaluru',
    'Karnataka',
    '560001',
    'Home'
);
```

6.3 Insert a Restaurant

```
INSERT INTO Restaurants (
    restaurant_name,
    cuisine_type,
    address,
    phone_number,
    rating
)
VALUES (
    'Food Palace',
    'Indian',
    'Anekal, Bengaluru',
```

```
    '9876501234',
    4.5
);
```

6.4 Insert a Menu Item

```
INSERT INTO MenuItems (
    restaurant_id,
    item_name,
    description,
    price,
    category
)
VALUES (
    1,
    'Veg Burger',
    'Fresh vegetable burger',
    120.00,
    'Main Course'
);
```

6.5 Insert an Order

```
INSERT INTO Orders (
    customer_id,
    restaurant_id,
    address_id,
    total_amount
```

```
)
VALUES (
    1,
    1,
    1,
    240.00
);
```

6.6 Insert Order Items

```
INSERT INTO OrderItems (
    order_id,
    item_id,
    quantity,
    price_at_order
)
VALUES (
    1,
    1,
    2,
    120.00
);
```

6.7 Insert a Delivery Agent

```
INSERT INTO DeliveryAgents (
    agent_name,
    phone_number,
```

```
    vehicle_number
)
VALUES (
    'Arun Kumar',
    '9123456780',
    'KA01AB1234'
);
```

6.8 Insert a Delivery Record

```
INSERT INTO Deliveries (
    order_id,
    agent_id
)
VALUES (
    1,
    1
);
```

6.9 Insert a Payment Record

```
INSERT INTO Payments (
    order_id,
    amount,
    payment_method
)
VALUES (
    1,
```

```
    240.00,
    'UPI'
);
```

6.10 Insert a Review

```
INSERT INTO Reviews (
    order_id,
    customer_id,
    restaurant_rating,
    delivery_rating,
    comment
)
VALUES (
    1,
    1,
    5,
    4,
    'Good food and delivery service'
);
```

Note: These sample statements assume that the database is empty and that the generated IDs begin at 1. If records already exist, use the actual IDs returned by MySQL. Do not execute these statements repeatedly without changing the unique values or checking existing records.

---

7. Verifying Relationships Between Tables

The following query retrieves information by joining related tables.

```
SELECT
```

```
    c.full_name,
    r.restaurant_name,
    o.order_id,
    o.order_status,
    o.total_amount,
    p.payment_method,
    p.payment_status
FROM Orders o
JOIN Customers c
    ON o.customer_id = c.customer_id
JOIN Restaurants r
    ON o.restaurant_id = r.restaurant_id
LEFT JOIN Payments p
    ON o.order_id = p.order_id;
```

Explanation:

- Customers is joined with Orders using `customer_id`.
- Restaurants is joined with Orders using `restaurant_id`.
- Payments is joined with Orders using `order_id`.
- LEFT JOIN allows an order to appear even if a payment record has not yet been created.

Expected Result: The query displays customer details, restaurant information, order status, order amount, and any associated payment details.

7.1 Verifying Foreign Key Definitions

```
SELECT
    TABLE_NAME,
    COLUMN_NAME,
    REFERENCED_TABLE_NAME,
    REFERENCED_COLUMN_NAME
```

```
FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = DATABASE()
AND REFERENCED_TABLE_NAME IS NOT NULL;
```

Expected Result: The output lists the foreign keys defined in the selected database.

7.2 Verifying Table Structures

```
SHOW TABLES;
```

This command displays all ten tables in FoodDeliveryDB.

To inspect a particular table:

```
DESCRIBE Customers;
DESCRIBE Orders;
DESCRIBE Payments;
DESCRIBE Reviews;
```

Expected Result: The output displays each table's columns, data types, and key information.

8. Summary of Constraints Used

Constraint	Purpose	Example
PRIMARY KEY	Uniquely identifies each record.	customer_id
FOREIGN KEY	Maintains relationships between tables.	Orders.customer_id
UNIQUE	Prevents duplicate values.	Customers.email

CHECK	Validates values against conditions.	CHECK (price >= 0)
NOT NULL	Prevents missing required values.	full_name
DEFAULT	Assigns an initial value automatically.	order_status DEFAULT 'Placed'

9. Result

Data integrity constraints were implemented and examined for the Online Food Delivery System.

Primary keys provide unique identification, foreign keys maintain relationships between tables, UNIQUE constraints prevent duplicate values, CHECK constraints validate data, NOT NULL constraints ensure that required fields are populated, and DEFAULT constraints provide consistent initial values.

SQL statements were provided to test invalid insertions, verify table structures, inspect foreign keys, and retrieve related data using JOIN operations.

The expected behavior is that valid records are accepted and records violating enforced constraints are rejected.

10. Conclusion

This assignment demonstrates the importance of data integrity constraints in a relational database. By applying primary keys, foreign keys, UNIQUE, CHECK, NOT NULL, and DEFAULT constraints, the Online Food Delivery System can maintain accurate and consistent data.

Foreign keys prevent invalid relationships, while UNIQUE and CHECK constraints enforce important business rules. DEFAULT constraints simplify data entry, and primary keys ensure that each record can be uniquely identified.

The verification queries help confirm that the constraints and relationships are defined correctly. Therefore, implementing appropriate data integrity constraints is essential for building a reliable, consistent, and maintainable database system.